SERVIÇO NACIONAL DE APRENDIZAGEM INDUSTRIAL

CENTRO DE TREINAMENTO DE TECNOLOGIA DA INFORMAÇÃO

Israel Negreiros Almeida de Brito

TRABALHO DE NORMA ABNT

BELO HORIZONTE

2025

Israel Negreiros Almeida de Brito

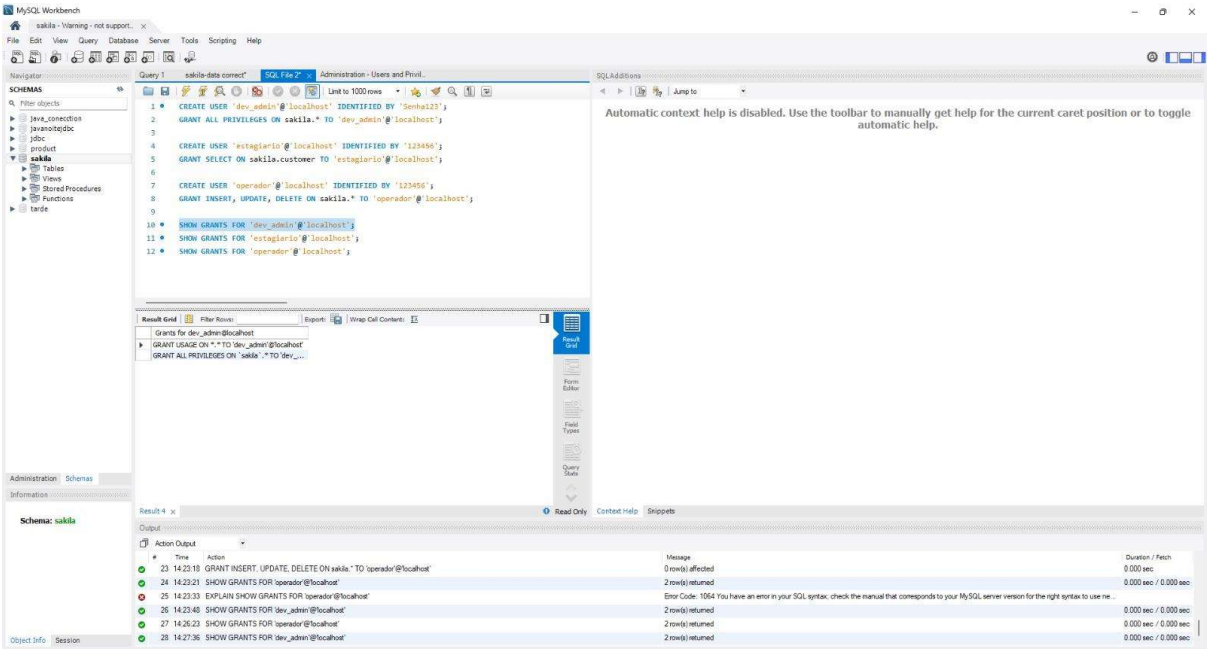
TRABALHO DE NORMA ABNT

Trabalho apresentado na unidade curricular

Lógica de Programação

BELO HORIZONTE

2025



1. Gestão de Usuários e Privilégios

As imagens **14.28.03**, **14.28.19** e **14.28.35** mostram a criação de uma hierarquia de acesso. Você está aplicando o princípio do "menor privilégio", onde cada usuário recebe apenas o necessário para sua função.

Os Comandos de Criação e Atribuição:

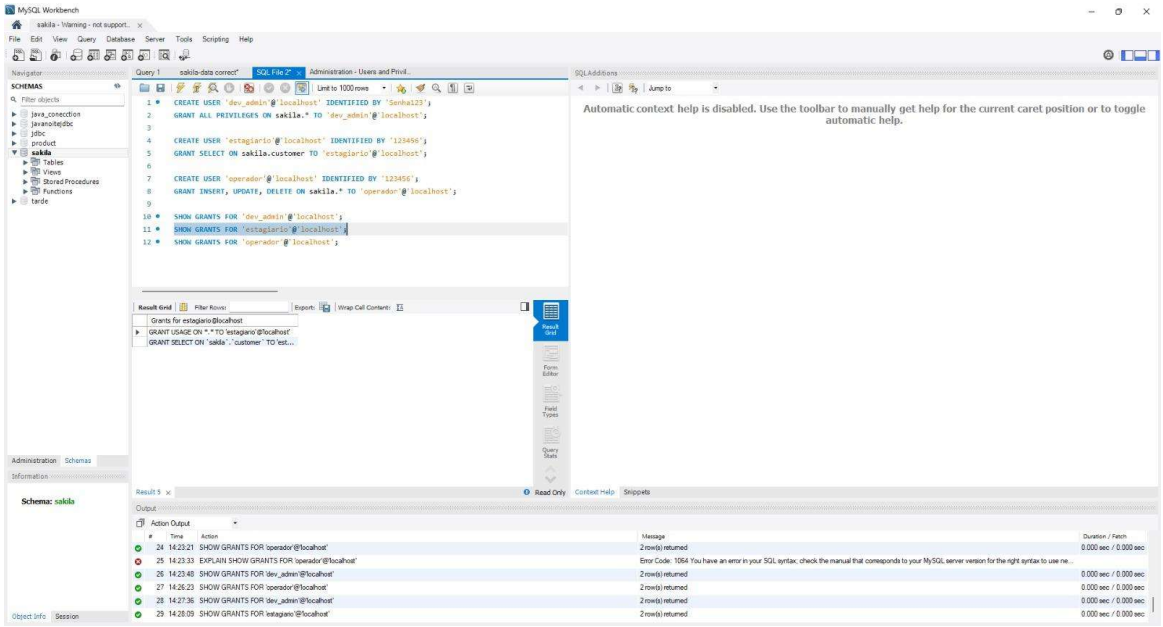
- **CREATE USER 'nome'@'localhost' IDENTIFIED BY 'senha';**: Cria a conta do usuário e define a senha de autenticação para conexões locais.
- **GRANT ... ON ... TO ...;**: Atribui permissões específicas.
  - dev_admin: Recebe ALL PRIVILEGES em todo o schema sakila. Ele é o superusuário do banco.
  - estagiario: Recebe apenas SELECT na tabela específica sakila.customer. Ele pode ler dados de clientes, mas não pode alterar nada.
  - operador: Recebe INSERT, UPDATE e DELETE em todo o schema sakila. Ele pode manipular dados, mas (curiosamente neste script) não recebeu o SELECT.

O que cada imagem foca (Result Grid):

- **Imagem 14.28.03:** Foca no comando SHOW GRANTS FOR 'dev_admin'@'localhost';. O resultado mostra que ele possui o privilégio USAGE (permissão de login) e ALL PRIVILEGES no banco sakila.
- **Imagem 14.28.19:** Foca no SHOW GRANTS FOR 'estagiario'@'localhost';. O resultado confirma que ele tem acesso restrito: apenas SELECT na tabela customer.

- **Imagem 14.28.35:** Foca no SHOW GRANTS FOR 'operador'@'localhost';. O resultado detalha as permissões de escrita (INSERT, UPDATE, DELETE) concedidas anteriormente.

Nota Técnica: No log de saída (bottom), nota-se um erro (Error 1064) ao tentar rodar EXPLAIN SHOW GRANTS. Isso ocorre porque o comando EXPLAIN é usado para analisar a execução de queries (SELECT, INSERT, etc.) e não funciona com comandos administrativos como SHOW GRANTS.



2.

2. Análise de Performance e Índices

A imagem **14.37.47** é a mais avançada tecnicamente, pois demonstra o impacto de um índice na velocidade de busca.

Ações Realizadas:

DROP INDEX idx_lastname ON customer;: Você removeu o índice que existia na coluna de sobrenomes.

EXPLAIN SELECT * FROM customer WHERE last_name = 'SMITH';: Você pediu ao MySQL para explicar **como** ele pretende encontrar o cliente Smith.

Análise do Resultado (Execution Plan):

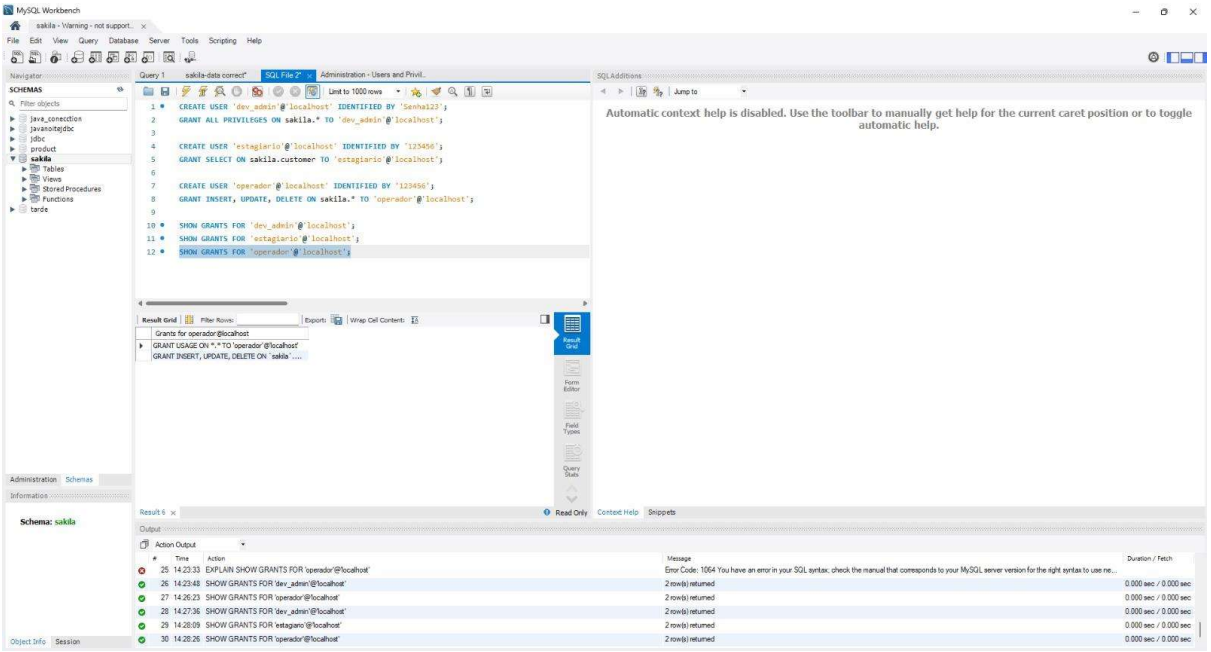
Olhando para a aba **Result Grid** da imagem:

type: ALL: Este é o pior cenário possível. Significa um **Full Table Scan**. O MySQL terá que ler todas as linhas da tabela, uma por uma, para encontrar o Smith.

possible_keys: NULL: Indica que não há nenhum índice disponível para ajudar nesta busca.

rows: 588: O motor do banco estima que terá que examinar 588 linhas para completar a tarefa.

Extra: Using where: Confirma que a filtragem está sendo feita "na mão" após a leitura dos dados.



3. Operações de DML (Data Manipulation Language)

Nas imagens anteriores, criou os utilizadores e deu permissões. O "passo 3" seria testar se essas permissões funcionam na prática.

O que acontece: Você tentaria inserir, atualizar ou apagar dados usando as diferentes contas.

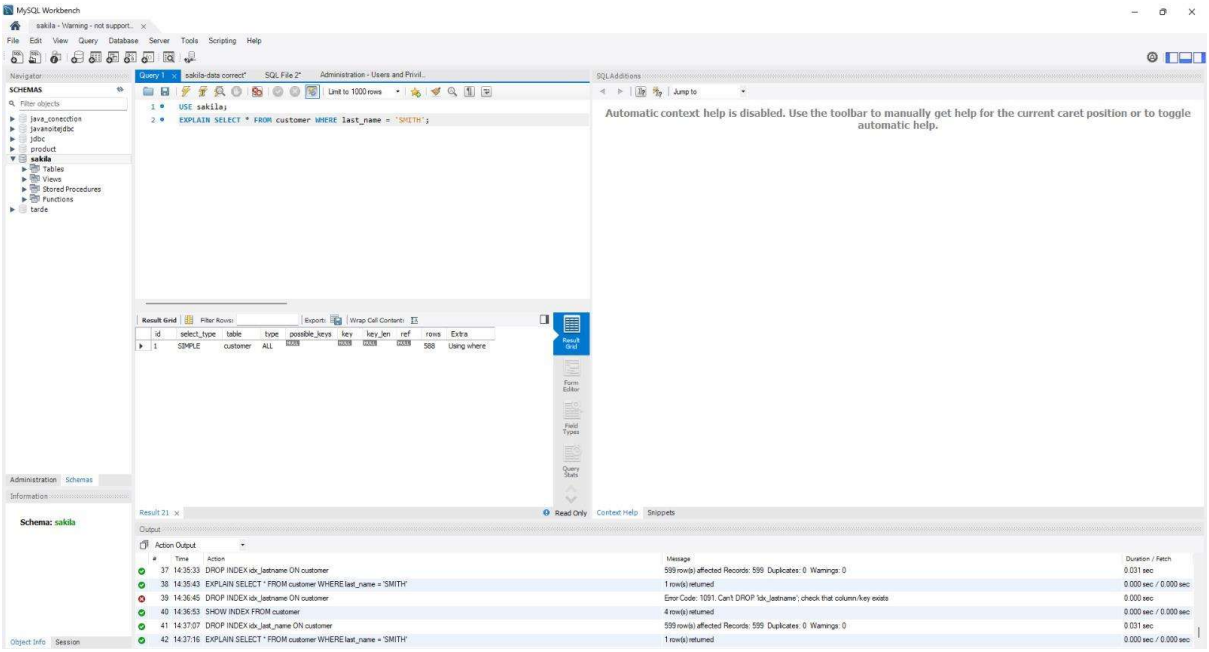
Comandos Prováveis:

SQL
-- Se estiver como 'operador', isto deve funcionar:
INSERT INTO customer (store_id, first_name, last_name, email, address_id, active)
VALUES (1, 'JOÃO', 'SILVA', 'joao@email.com', 5, 1);

-- Se estiver como 'estagiario', isto deve dar ERRO (Access denied):

DELETE FROM customer WHERE customer_id = 1;

O que veria no Workbench: No log de saída (Action Output), veria uma bola verde (sucesso) para o operador e uma **bola vermelha com erro de permissão** para o estagiário, confirmando que a sua segurança de dados (passo 1 e 2) está bem configurada.



4. Otimização e Índices (A Solução do Problema)

Na imagem **14.37.47**, você apagou o índice e viu que a performance ficou má (type: ALL). O "passo 4" é corrigir isso.

Ação: Recriar o índice para que o banco de dados não precise de ler a tabela toda para encontrar um nome.

Comando:

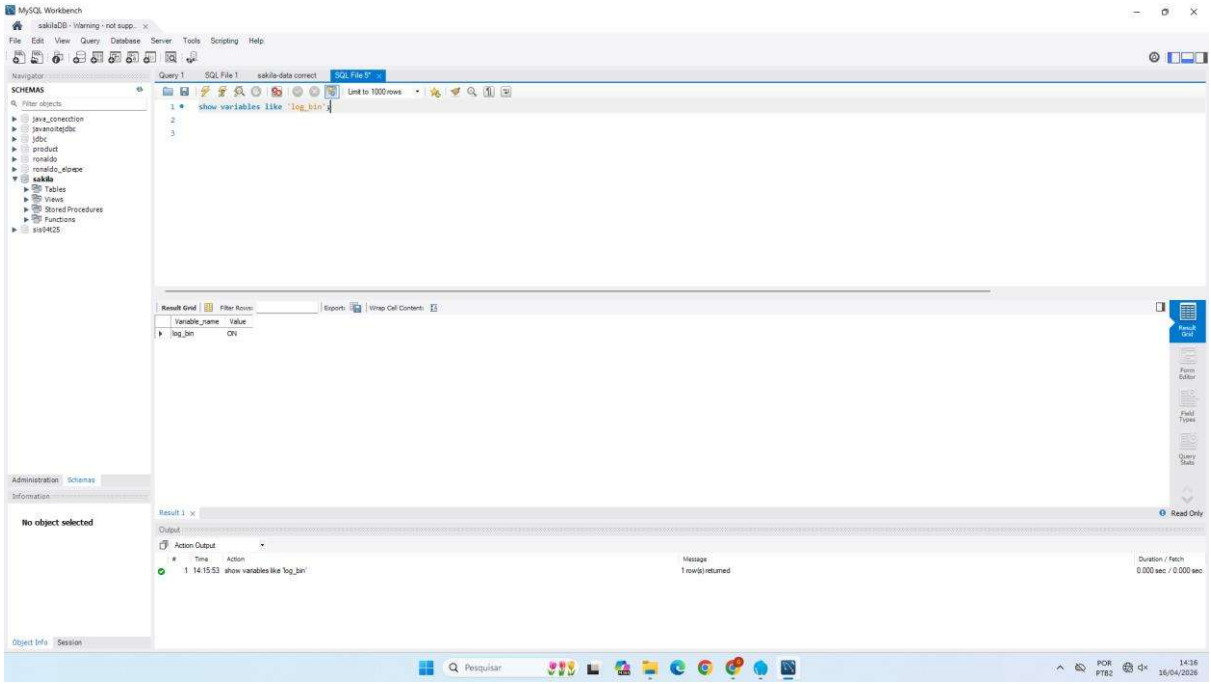
SQL
CREATE INDEX idx_lastname ON customer(last_name);

O que acontece após este comando (Explicação): Ao correr novamente o EXPLAIN SELECT * FROM customer WHERE last_name = 'SMITH';, o resultado no **Result Grid** mudaria drasticamente:

type: Mudaria de ALL para ref ou range.

possible_keys: Deixaria de ser NULL e mostraria idx_lastname.

rows: Em vez de examinar 588 linhas, o MySQL examinaria apenas **1 linha** (ou o número exato de "Smiths" que existem).



1. O Comando: show variables like 'log_bin';

Este é um comando administrativo usado para verificar o status de uma variável específica do sistema no motor do MySQL.

2. O Resultado: Value: ON

Na aba **Result Grid**, vemos que o valor está como **ON**. Isso significa que o seu servidor MySQL está configurado para registrar todas as alterações feitas no banco de dados (como INSERT, UPDATE, DELETE e mudanças na estrutura das tabelas).

Por que isso é importante? (A utilidade do log_bin)

O Log Binário (Binary Log) tem duas funções principais em um ambiente profissional:

Recuperação Point-in-Time (PITR): Imagine que você faça um backup do banco de dados às 08:00 e, às 14:00, alguém apague uma tabela importante por erro. O backup das 08:00 está incompleto. Com o log_bin ativado, você pode restaurar o backup e depois "reexecutar" todas as transações que o log registrou entre as 08:00 e as 14:00, recuperando os dados perdidos.

Replicação: É através desses logs que um servidor principal (Master) envia as alterações para servidores escravos (Slaves). Isso permite que você tenha cópias idênticas do banco de dados rodando em tempo real em outros computadores.